
Assignment 3

COMP 2402 AB - Fall 2021

Assignment #3

Due: Sunday November 7, 23:59

Submit early and often. No late submissions will be accepted.

Academic Integrity

You may:

- Discuss general approaches with course staff,
- Discuss general approaches with your classmates,
- Use code and/or ideas from the textbook,
- Use a search engine / the internet to look up basic Java syntax,
- Send your code / screen share your code with course staff (although it is unlikely the course staff has the bandwidth to look at individuals' code, unfortunately.)

You may not:

- Send or otherwise share code or code snippets with classmates,
- Use code not written by you, unless it is code from the textbook (and you should cite it in comments),
- Use a search engine / the internet to look up approaches to the assignment,
- Use code from previous iterations of the course, unless it was solely written by you,
- Use the internet to find source code or videos that give solutions to the assignment.

If you ever have any questions about what is or is not allowable regarding academic integrity, please do not hesitate to reach out to course staff. We will be happy to answer. Sometimes it is difficult to determine the exact line, but if you cross it the punishment is severe and out of our hands. Any student caught violating academic integrity, whether intentionally or not, will be reported to the Associate Dean and be penalized as follows:

- First offence: F in the course.
- Second offence: One-year suspension from program.
- Third offence: Expulsion from the University.

These are standard penalties. More-severe penalties will be applied in cases of egregious offences. For more information, please see Carleton University's [Academic Integrity](#) page.

Assignment 3

Grading

This assignment will be tested and graded by a computer program (and **you can submit as many times as you like, your highest grade is recorded**). For this to work, there are some important rules you must follow:

- Keep the directory structure of the provided zip file. If you find a file in the subdirectory `comp2402a3` leave it there.
- Keep the package structure of the provided zip file. If you find a package `comp2402a3`; directive at the top of a file, leave it there.
- Do not rename or change the visibility of any methods already present. If a method or class is public leave it that way.
- Do not change the `main(String[])` method of any class. Some of these are setup to read command line arguments and/or open input and output files. Don't change this behaviour. Instead, learn how to use command-line arguments to do your own testing.
- Submit early and often. The submission server compiles and runs your code and gives you a mark. You can submit as often as you like and only your latest submission will count. There is no excuse for submitting code that does not compile or does not pass tests.
- Write efficient code. The submission server places a limit on how much time it will spend executing your code, even on inputs with a million lines. For some questions it also places a limit on how much memory your code can use. If you choose and use your data structures correctly, your code will easily execute within the time limit. Choose the wrong data structure, or use it the wrong way, and your code will be too slow for the submission server to grade (resulting in a grade of 0).

Submitting and Server Tests

The submission server is ready: [here](#). If you have issues, please post to Discord to the teaching team (or the class) and we'll see if we can help.

When you submit your code, the server runs tests on your code. These are bigger and better tests than the small number of tests provided in the "Local Tests" section further down this document. They are obfuscated from you, because you should try to find exhaustive tests of your own code. This can be frustrating because you are still learning to think critically about your own code, to figure out where its weaknesses are. But have patience with yourself and make sure you read the question carefully to understand its edge cases.

Warning: Do not wait until the last minute to submit your assignment. There is a hard 5 second limit on the time each test has to complete. If the server is heavily loaded, borderline tests may start to fail. **You can submit multiple times and your best score is recorded.**

Changes from Assignment 2's Server: (as best we can manage...)

- For Part C, it will now continue running tests even if you fail an earlier one. We were doing this for Part B already.

Assignment 3

- For all parts, each test is now annotated with its purpose (correctness, edge case, or performance.)
- Generally, less than half the tests are for correctness (including edge cases). The remaining half is for performance, that is, are you using a good data structure for time and space? Remember, this course's focus is on improving performance so to pass the part, you have to show you are using some of the concepts we are talking about in this course.
- The error messages try to guess what is going on a bit more, and tries to make suggestions on how you might improve based on the information it has. The script only has so much information, though, so it can only do so much (and sometimes it guesses wrong, as you've seen.)

The Assignment

Purpose & Goals

Generally, the assignments in this course are meant to give you the opportunity to practice with the topics of this course in a way that is challenging yet also manageable. At times you may struggle and at others it may seem more straight-forward; just remember to keep trying and practicing, and over time you will improve.

Specifically, assignment 3 aims to improve your understanding of the **linked-list** data structures we've been talking about by:

- Encouraging you to think about how you need to use and access data in your algorithm before choosing the data structure(s) for your data, with a focus on thinking about which of a [List](#) or a [SSet](#) is more appropriate, in a variety of problems,
- Seeing the value of [ListIterators](#) for linked lists, and
- Implementing parts of the List interface using DLList and SkiplistList as the backing data type, using some of the concepts we've been learning about. Hopefully you will contrast these solutions with the array-based data types to see where they must be different.

Part A: The Setup

Start by downloading and decompressing the Assignment 3 Zip File (comp2402a3.zip), which contains a skeleton of the code you need to write. You should have the files: Part0.java, Part1.java, Part2.java, Part3.java, Part4.java, MyList.java, MyDLList.java, SkiplistList.java, and Tester.java.

As with assignments 1 & 2, you can test Parts0-4 using the command line or input/output files. You can test Part C with the Tester class included in the zip file. As before, you'll want to add your own tests as this is just given as a basis on which you should build. The submission server will do thorough testing; it is your job to read the specifications as carefully as possible, and to examine your own code for potential problems and test those cases.

If you are having trouble running these programs, as given, **figure this out first before attempting to do the assignment**. Check the assignment 3 FAQ on discord and ask questions if you are stuck and course staff or another student will likely help you fairly quickly.

Assignment 3

Part B: The Interface

As with assignments 1 and 2, you will be filling in the `doIt()` method in each of the appropriate `PartX` classes. You can go back to assignments 1 and 2 as reference.

You can download some (very limited) sample input and output files for each question as a [zip file \(a3-io.zip\)](#). If you find that a particular question is unclear, you can possibly clarify it by looking at the sample files for that question. **Note that these are very limited tests; you can and should add to them to test your code more thoroughly** as the server tests are much more extensive.

Unless specified otherwise, "sorted order" refers to the natural sorted order on Strings, as defined by `String.compareTo(s)`.

1. [5 marks] (Assignment 2.3 Redux) Read the input one line at a time, and output the last x **distinct** lines in sorted order, where $x \geq 0$ is a parameter to the `doIt` method. If there are fewer than x lines, print the (less than x) existing distinct lines. For example, input lines "a", "b", "b", "c" and $x=3$ would output "a", "b", "c", as would the input "a", "b", "c", "b". As another example, input lines "x", "e", "b", "e", "c", "d" with $x=3$ would output "c", "d", "e". As a last example, input lines "a", "b", "c", "a", "d" and $x=3$ would output "a", "c", "d". To get full points, try to use as little extra space and time as possible.
2. [7 marks] Read in the input one line at a time and output a line if it is the maximum over the last x lines, where $x \geq 0$ is a parameter to the `doIt` method. That is, output a line if there are at least x lines total, and this line is the maximum of the $x-1$ lines previous to the current one and the current one. For example, input lines "y", "a", "b", "p", "p" with $x=3$ would output "p", "p" since p is the maximum of $\{a, b, p\}$ and p the second p is the maximum of $\{b, p, p\}$. We don't print out y because there are not 2 other elements to compare it to when we read it in. Note that with this definition, when $x=0$ every line is a maximum. As another example, input lines "x", "e", "b", "e", "c", "d" with $x=3$ would output "e". To get full points, try to use as little extra space and time as possible.
3. [8 marks] Read in the input one line at a time and output a new line if it is a prefix of some previous line. For example, input lines "abc", "def", "abc", "de", "ef", "efg" would output "abc", "de" because they are prefixes of lines that come before them. The `String.startsWith()` method may be helpful here. To get full points, try to use as little extra space and time as possible.
4. [10 marks] Read in the input one line at a time and output a new line if it is a substring of some previous line. To get full points, try to use as little extra space and time as possible. For example, input lines "abc", "def", "abc", "de", "ef", "efg" would output "abc", "de", "ef" because they are

Assignment 3

all substrings of lines that come before them. To get full points, try to use as little extra space and time as possible.

Hint: A string u is a substring of string s if it is a prefix of some suffix of s .

Note: Getting full points here may be challenging! A string of length k has $k+1$ suffixes (including the empty suffix), that combine to have lengths $0+1+2+\dots+k=O(k^2)$. To pass the last performance test, you will have to think of a better way of storing suffixes.

Note: the JCF `SortedSet` interface doesn't have an explicit `find(x)` method; rather, you have to combine the `tailSet(x)` and `first()` methods.

Note: You can use more than one data structure to store your data. As long as you're not increasing the space by more than a constant factor, you should be fine. (As in, having two data structures of size s is still asymptotically the same as having one data structure of size s .)

Part C: The Implementation

Files Provided

The starter code contains the `DLList` and `SkiplistList` classes, which are implementations of the provided `MyList` interface. Part C involves you completing implementations of 5 methods within these two classes, most of which will be familiar to you. These are described in more detail below.

Your Tasks

1. [5 marks] (Assignment 1.4 Redux) In the `DLList` class, implement

```
public MyList<T> shuffle(MyList<T> other)
```

Return a new `DLList` that results from alternating individual elements from this and other, as with the shuffle on assignment 1.

If the given lists are not of equal length, just place the extra elements onto the end of the returned `DLList`.

2. [5 marks] (Assignment 2.C Redux) In the `DLList` class, implement

```
public void shrink()
```

that "shrinks" the `DLList` by repeatedly removing pairs of consecutive duplicates. For example, if the list is `[c,a,c,c,a,b,b,b]` then `shrink` will make the list be `[c,b]`. Note that the size of the list changes from 8 down to 2.

Assignment 3

3. [10 marks] In the DLList class, implement

```
public MyList<T> chop(int i)
```

that chops our list into two parts at index i . The DLList this contains elements from indices $0, \dots, i-1$, and the returned list is a DLList that contains elements from indices $i, \dots, n-1$. This should work for all indices from 0 to n , where n is the size of this.

For example, suppose `list=[0,1,2,3]`. Then `list.chop[3]` returns `[3]` and list becomes `[0,1,2]`.

Note: The DLList class provides a built-in [ListIterator](#) that iterates through a DLList. You can instantiate such an Iterator with code like:

```
// get an iterator that initially is at index 0
ListIterator<T> it1 = new Iterator(this, 0);

// get an iterator over the DLList variable, initially at index i
ListIterator<T> it2 = new Iterator(<a DLList variable>, i);
```

4. [10 marks]-(Assignment 1.6 Redux) In the SkiplistList class, implement

```
public void reverse()
```

Reverse the SkiplistList without using recursion or too much extra space.

Hint: How would you reverse a SLList? Start there then build up.

Note: To get all the points and glory, try to implement this in $O(1)$ space and $O(n \log n)$ time.

5. [15 marks] In the SkiplistList class, implement

```
public MyList<T> chop(int i)
```

that chops our list into two parts at index i , as in 7.

Note: The SkiplistList class provides a built-in Iterator that iterates through a SkiplistList. You can instantiate such an Iterator with code like:

```
Iterator<T> it = this.iterator();
```

Local Tests

For Part B, you can download some sample input and output files for each question as a [zip file \(a3-io.zip\)](#). Once you get a sense for how to test your code with these input files, write your own tests that test your program more thoroughly (**the provided tests are not exhaustive!**)

For Part C (DLList, SkiplistList), a Tester class is provided with some tests of the provided methods. I recommend you add tests to Tester.java to test your DLList and SkiplistList locally (as opposed to on the server).

Assignment 3

Testing suggestions:

- Test incrementally, that is, bit-by-bit,
- Test edge cases, e.g. empty files, empty lists, parameters of odd or even sizes,
- Use small test cases at first, then go bigger only when necessary,
- Test individual methods as you write them, even. As in: you can test an incomplete method to make sure it's doing what you're expected thus far.

Tips, Tricks, and FAQs

How should I approach each problem?

- Make sure you understand it. Construct small examples, and compute (by hand) the expected output. If you aren't sure what the output should be, go no further until you get clarification.
- Now that you understand what you are supposed to output, and you've been able to solve it by hand, think about how you solved it and whether you could explain it to someone. How about explaining it to a computer?
- If it still seems challenging, what about a simpler case? Can you solve a similar or simplified problem? Maybe a special case? If you were allowed to make certain assumptions, could you do it then? Try constructing your code incrementally, solving the smaller or simpler problems, then, only expanding scope once you're sure your simplified problems are solved.

How should I test my code?

- You should be testing your code as you go along.
- Start small so that you can compute the correct solution by hand.
- Think about edge cases.
- Think about large inputs, random inputs.
- Test for speed.